

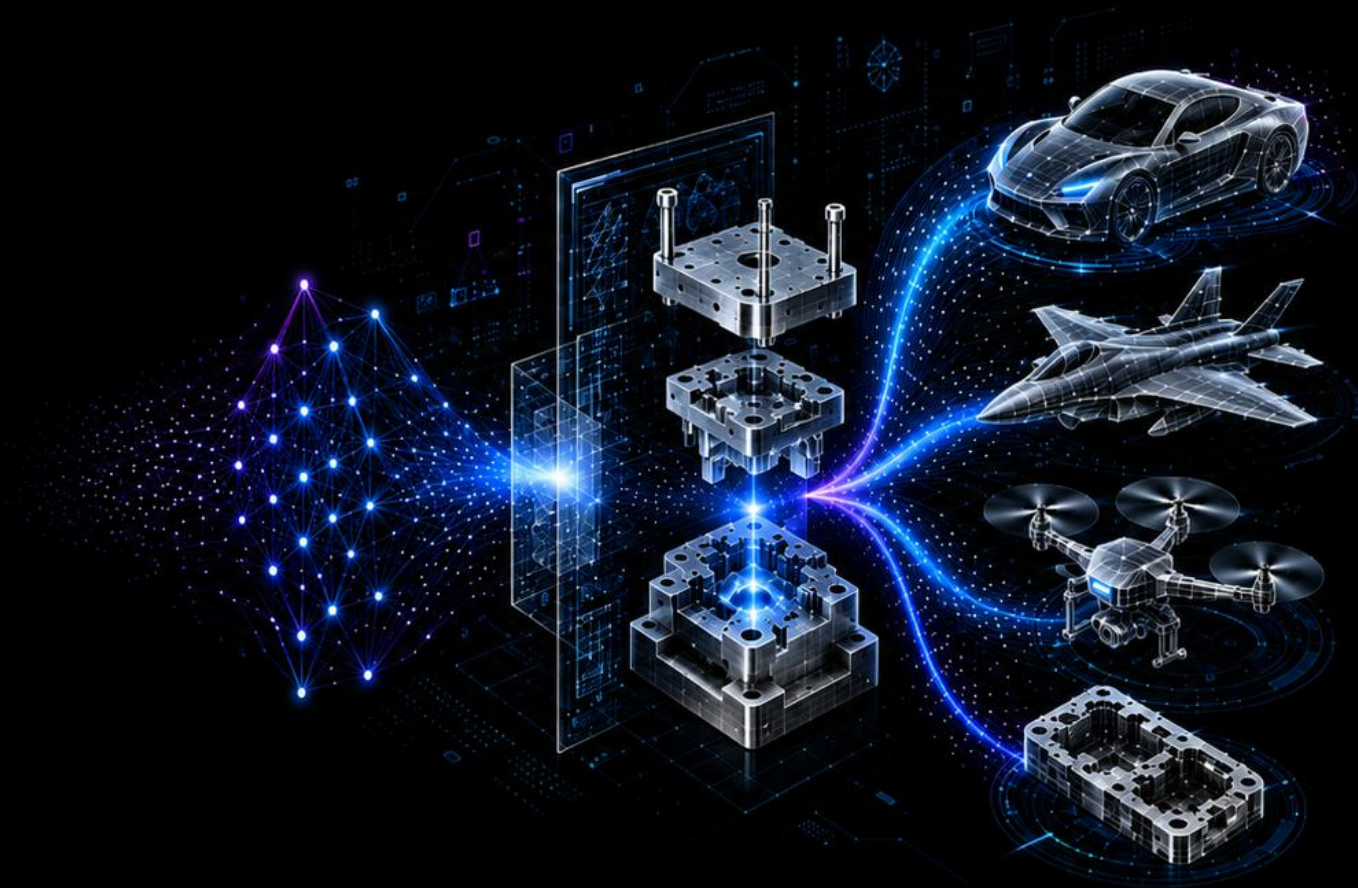
개인 과제 발표

# Text2CAD - 3

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design  
for Engineering  
Applications Lab

## ■ 진행 상황

### 연구 피드백 과제

- ✓ text → CAD가 되는 전 과정들을 더 자세히 정리해서 설명
- ✓ 각 문제 상황들과 그 해결책들을 이해하기 쉽게 정리해서 설명  
(설명 시 파일 사진도 첨부)
- ✓ Demo 영상 간결하게 다시 찍기
- ✓ Powershell에서 prompt를 작성하는 것이 아닌,  
input file로 prompt를 넣는 방식으로 변경

### CAD Agent 논문 과제

- ✓ 논문을 LLM 통해 읽어본 후,
  - "해당논문이 어떤 문제를 해결 하고자 했으며, 어떻게 해결했는지 파악
  - 본인 연구는 어떻게 차별성을 가지면 좋을지 고민

## ■ 연구의 전체적인 구조

### 전체 연구 과정 정리

- ✓ 해당 연구는 단순히 “LLM에게 FreeCAD 코드를 작성 시켜 도면을 만든다”에서 시작했지만,

발전 시켜 나가는 과정에서 정확성·재현성·오류 복구 문제가 계속 발견되면서 오른쪽 사진과 같은 구조로 발전

⇒ 핵심 변화는 LLM에게 모든 일을 맡기던 구조에서, LLM은 자연어 해석만 담당하고 정확한 계산·코드 생성·검증은 프로그램이 담당하도록 역할을 분리

### 초기 방식

자연어 → LLM → FreeCAD Python 코드 → 실행

### 현재 방식

자연어

- 요구사항 목록 추출
- LLM이 제한된 JSON 설계 계획 생성
- 스키마 검증
- 원문 기반 치수 재검증·교정
- 결정론적 Python 컴파일
- FreeCAD 실행
- 실제 형상 검증
- FCStd / STEP / STP 출력

## 1. 전체 구조 변화 요약

✓ 전체 구조 변화는 다음과 같이 총 5번의 문제 상황을 거쳐 지금의 시스템으로 발전하였다.

| 단계    | 구조                            | 발견된 핵심 문제                 | 개선 방향                |
|-------|-------------------------------|---------------------------|----------------------|
| 초기    | 자연어 → LLM → Python 코드         | 코드가 매번 달라지고 실행 오류 발생      | 프롬프트 강화              |
| 1차 개선 | 강한 규칙 프롬프트 → Python 코드        | 복잡해질수록 규칙이 끝없이 증가         | LLM의 코드 생성을 제거       |
| 2차 개선 | 자연어 → JSON → Python           | JSON 항목 누락·형상 오분류         | 요구사항 목록과 계획 생성 분리    |
| 3차 개선 | 요구사항 목록 → JSON 계획 → 최대 3회 수정  | 잘못된 계획을 자동 복구할 필요         | 스키마 검증·복구 루프         |
| 4차 개선 | Qwen3:8B + JSON 계획 + 결정론적 컴파일 | LLM이 코드를 직접 생성하면 재현성 부족   | 모델 역할 분리             |
| 5차 개선 | 원문 → JSON → 원문 숫자 재추출·비교      | 60 이 6 으로 바뀌어도 스키마 통과     | 의미·치수 검증 추가          |
| 현재    | 계획·검증·컴파일·CAD·형상검증 분리         | 복잡한 자유곡면과 의미 검증은 계속 확장 필요 | 기능별 DSL과 결정론적 생성기 확장 |

## 2. 최초 방식: LLM이 FreeCAD Python 코드를 직접 생성

### 초기 구조

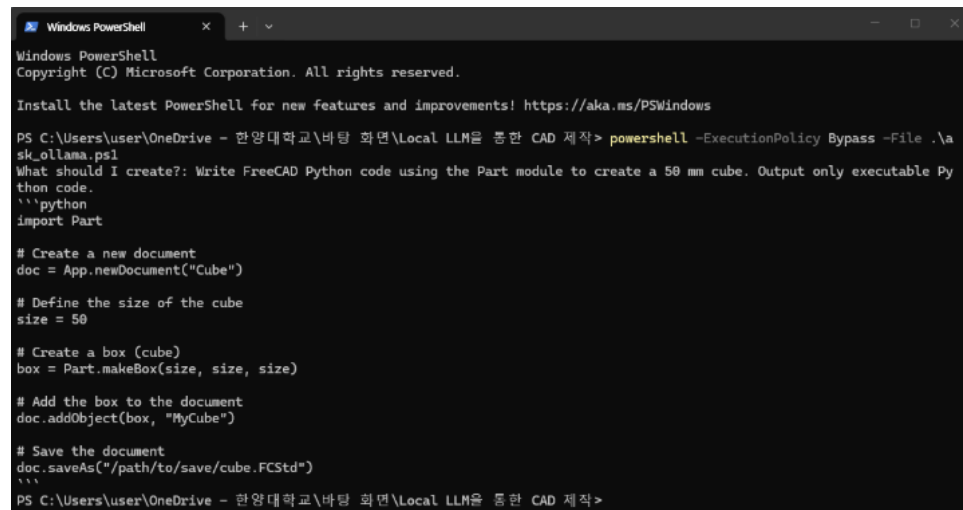
```
사용자 자연어  
→ ask_ollama.ps1  
→ Ollama  
→ Qwen2.5-Coder:7B  
→ FreeCAD Python 코드  
→ generated.py  
→ FreeCAD 실행
```

1. Powershell 에 ask\_ollama.ps1 코드 입력
2. 자연어를 입력
3. 첫 번째 코드를 통해 Ollama(LLM 실행 환경) API 호출
4. Ollama를 통해 Qwen 모델을 실행
5. Qwen이 자연어 요청을 해석하고 FreeCAD Python 코드 생성
6. 코드를 generated.py로 저장
7. FreeCAD 실행

### 최초 성공

- ✓ LLM에 아래의 자연어 요청

Write FreeCAD Python code using the Part module to create a 50 mm cube.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> powershell -ExecutionPolicy Bypass -File .\ask_ollama.ps1
What should I create?: Write FreeCAD Python code using the Part module to create a 50 mm cube. Output only executable Python code.
'''python
import Part

# Create a new document
doc = App.newDocument("Cube")

# Define the size of the cube
size = 50

# Create a box (cube)
box = Part.makeBox(size, size, size)

# Add the box to the document
doc.addObject(box, "MyCube")

# Save the document
doc.saveAs("/path/to/save/cube.FCStd")
'''
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작>
```

⇒ 완전한 시스템은 아니지만 Local 모델과 FreeCAD를 연결할 수 있다는 가능성 확인.

## 3. 최초 문제: LLM이 만든 코드가 실행 가능한 FreeCAD 코드라는 보장이 없음

### 실제 문제

- ✓ 단순한 큐브 형상을 요청했음에도 필요 이상으로 복잡한 방식을 사용  
(예) 다음 처럼 큐브 꼭짓점과 면을 직접 구성하려고 함

```
cube_vertices = [...]  
cube_faces = [...]  
cube = Part.makePolygon(...)  
cube = Part.Face(cube).extrude(...)
```

하지만, 단순 직육면체는 다음과 같은 코드 한줄이면 완성 가능

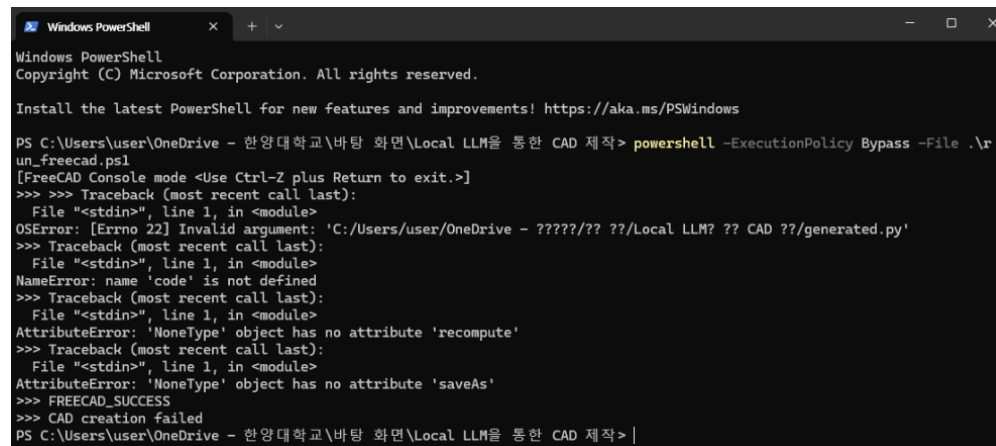
```
shape = Part.makeBox(50, 50, 50)
```

- ✓ 위와 같이 불필요하게 복잡한 코드를 사용하면 생기는 문제점
  - 잘못된 Face 생성
  - 중복된 점
  - 잘못된 API 호출
  - 같은 요청 임에도 다른 코드 생성
  - 저장 경로를 임의로 생성

### 실행 과정에서도 오류 발생

- ✓ 초기 run\_freecad.ps1은 OneDrive 경로를 FreeCAD 콘솔에 문자열로 넘기면서 경로가 깨짐.
- ⇒ 실행 코드가 FreeCAD 에 제대로 전달되지 않아서 다음 오류가 연쇄적으로 발생.

- `OSError: Invalid argument`
- `NameError: code is not defined`
- `AttributeError: NoneType object has no attribute recompute`
- `AttributeError: NoneType object has no attribute saveAs`



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> powershell -ExecutionPolicy Bypass -File .\run_freecad.ps1
[FreeCAD Console mode <Use Ctrl-Z plus Return to exit.>]
>>> Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OSError: [Errno 22] Invalid argument: 'C:/Users/user/OneDrive - ?????/?? Local LLM? ?? CAD ??/generated.py'
>>> Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'code' is not defined
>>> Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'NoneType' object has no attribute 'recompute'
>>> Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'NoneType' object has no attribute 'saveAs'
>>> FREECAD_SUCCESS
>>> CAD creation failed
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작>
```

## ■ 3. 최초 문제: LLM이 만든 코드가 실행 가능한 FreeCAD 코드라는 보장이 없음

### 해결 과정

✓ 실행 구조를 변경해서 FreeCAD가 다음을 수행

1. generated.py를 정확한 경로에서 읽고 실행
2. FreeCAD 환경 내부에서 코드 실행
3. FreeCAD에서 Result 형상 생성
4. result.FCStd로 CAD 파일 저장
5. 성공 메시지 출력

❖ **result.FCStd** 는 이미지나 Python 코드가 아닌 FreeCAD 프로젝트 및 3D 형상을 저장한 원본 CAD 문서

### 해당 과정에서 배운 점

✓ 단순히 LLM이 Python 문장을 출력했다고 해서 CAD가 정확히 생성된 것은 아니다.

✓ 최소한 다음이 모두 확인되어야 한다.

#### 코드 생성

- ≠ 문법적으로 올바른 코드
- ≠ FreeCAD에서 실행 가능한 코드
- ≠ 올바른 solid
- ≠ 사용자 치수와 일치하는 결과

## 4. 프롬프트 강화 방식: LLM 코드 오류를 줄이기 위한 해결책

### 이유와 방법

#### ✓ 이유:

해당 LLM은 FreeCAD 코드를 “가능성 높은 텍스트”로 생성했기 때문에, 정확한 API 사용을 반드시 보장하지 않았다.

#### ✓ 방법: 따라서 프롬프트로 다음을 강제

(예) LLM 코드 오류를 줄이기 위해 ask\_ollama.ps1의 프롬프트에 규칙을 추가

```
$generatorPrompt = @"  
Generate complete executable Python code for FreeCAD 1.1.  
"
```

#### Mandatory rules:

- Always use Part.makeBox() for rectangular boxes.
- The first argument of doc.addObject must be "Part::Feature".
- Do not use makePolygon for a rectangular box.
- Do not save or close the document.
- Output executable Python code only.
- Do not include Markdown fences or explanations.

### ask\_ollama.ps1 파일의 의미

- ✓ 사용자와 Ollama/Qwen 사이에서 요청과 응답을 전달하는 연결 및 제어 프로그램 파일

#### [내부 구조]

: 오른쪽 사진 바로 아래에  
**generatorPrompt = @"**  
적혀있다.

```
$request = Read-Host "What should I create?"  
  
$body = @{  
    model = "qwen2.5-coder:7b"  
    prompt = $request  
    stream = $false  
} | ConvertTo-Json  
  
$response = Invoke-RestMethod `   
    -Uri "http://127.0.0.1:11434/api/generate" `   
    -Method Post `   
    -ContentType "application/json" `   
    -Body $body
```

#### [각 부분의 의미]

- Read-Host : 사용자 요청 입력
- \$body: 모델 이름과 프롬프트 구성
- ConvertTo-Json : Ollama가 받을 JSON 형식으로 변환
- Invoke-RestMethod : Ollama의 로컬 API 호출
- 127.0.0.1:11434 : 현재 컴퓨터에서 실행 중인 Ollama 주소
- \$response: Qwen이 생성한 응답

## 4. 프롬프트 강화 방식: LLM 코드 오류를 줄이기 위한 해결책

### 효과

- ✓ 간단한 플레이트와 구멍은 안정적으로 생성되기 시작

(예) STEP 3 생성 단계

자연어 요청: Create one solid mechanical part in FreeCAD. Create a rectangular base plate with **X length 80 mm, Y width 50 mm, and Z thickness 5 mm**. Create a cylindrical boss with radius 15 mm and **height 10 mm**. Center the boss at **X=40 mm and Y=25 mm**. The boss must start on the top surface of the plate at **Z=5 mm** and extend in the positive Z direction. Fuse the boss and the base plate into one solid. Create a cylindrical through hole with diameter 8 mm. Center the hole at **X=40 mm and Y=25 mm**. **The hole must start at Z=0** and extend 15 mm in the positive Z direction. Subtract the hole from the fused solid. Store the final shape in one Part::Feature named Result.

```
base = Part.makeBox(80, 50, 5)
hole = Part.makeCylinder(10, 5, App.Vector(40, 25, 0))
shape = base.cut(hole)
```

### 한계

- ✓ 형상이 복잡해질 때마다 프롬프트 규칙을 계속 추가해야 함.

(예)

- 원통 방향 지정 규칙
- 중간 객체를 문서에 넣지 말라는 규칙
- 박스와 원통을 먼저 결합하라는 규칙
- 구멍은 결합 후 제거하라는 규칙
- 최종 객체만 **Result** 로 생성하라는 규칙
- ID 중복 방지 규칙

⇒ 결국 프롬프트가 점점 길어지고, 한 규칙을 추가하면 다른 규칙을 모델이 놓치는 문제가 생김.

## 5. 복잡한 형상에서의 LLM 직접 코드 생성 방식의 한계

```
FreeCAD.ps1 generated.py 2 X
ers > user > OneDrive - 한양대학교 > 바탕 화면 > Local LLM을 통한 CAD 제작 > generated.py > ...
import FreeCAD as App
import Part

doc = App.newDocument("GeneratedCAD")

# Create rectangular base plate
base_plate_shape = Part.makeBox(80, 50, 5)
base_plate_feature = doc.addObject("Part::Feature", "BasePlate")
base_plate_feature.Shape = base_plate_shape

# Create cylindrical boss
boss_radius = 15
boss_height = 10
boss_center = App.Vector(40, 25, 5)
boss_shape = Part.makeCylinder(boss_radius, boss_height)
boss_feature = doc.addObject("Part::Feature", "Boss")
boss_feature.Shape = boss_shape

# Fuse base plate and boss
fused_shape = base_plate_shape.fuse(boss_shape)
fused_feature = doc.addObject("Part::Feature", "Fused")
fused_feature.Shape = fused_shape

# Create cylindrical through hole
hole_radius = 4 / 2
hole_height = 15
hole_center = App.Vector(40, 25, 0)
hole_shape = Part.makeCylinder(hole_radius, hole_height)
hole_feature = doc.addObject("Part::Feature", "Hole")
hole_feature.Shape = hole_shape

# Subtract hole from fused solid
final_shape = fused_shape.cut(hole_shape)
result_feature = doc.addObject("Part::Feature", "Result")
result_feature.Shape = final_shape

doc.recompute()
```

### 문제 상황

- ✓ 오른쪽 사진에서 보이는 것과 같이 현재 모델은 중간 형상까지 FreeCAD 문서에 추가함

(예) 다음 객체를 각각 문서에 작성

- BasePlate
- Boss
- Fused
- Hole
- Result

하지만, 우리가 원하는 것은 최종 형상 생성만 요청한 상황.

## ■ 5. 복잡한 형상에서의 LLM 직접 코드 생성 방식의 한계

### 문제점

✓ 이렇게 되면, LLM에게 전체 코드를 모두 맡긴 상태가 되고  
아래 목록 중 하나만 틀려도 전체 코드가 실패하게 된다.

- 자연어 해석
- 형상 분류
- 정확한 치수 복사
- 변수 이름 생성
- FreeCAD API 선택
- 연산 순서 결정
- Boolean 연산
- 문서 객체 구성
- 최종 저장 구조

### 결론

⇒ “LLM에게 FreeCAD 코드를 직접 쓰게 하는 방식”은 실험용으로는 가능하지만,  
범용적이고 정교한 연구 시스템의 핵심구조로 발전시키기에는 부족하다고 판단

## 6. 첫 번째 큰 구조 변경: JSON 중간 모델 도입

### 새로운 구조

사용자 자연어 요청

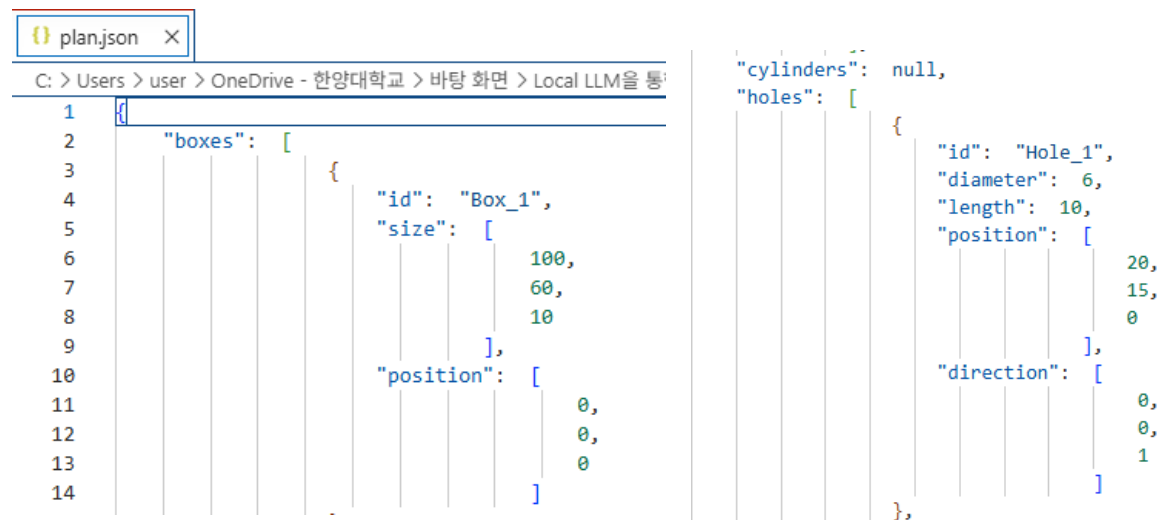
- PowerShell이 Ollama API를 호출
- Ollama에서 Qwen3:8B 실행
- Qwen이 JSON 설계 계획 생성
- compile\_plan.ps1(해당 코드를 Powershell에 다시 입력하면)이 JSON을 Python 코드로 변환
- run\_freecad.ps1 (CAD 실행하라는 프롬프트 명령 코드)
- FreeCAD가 Python 코드를 실행
- CAD 파일 생성

- ✓ Ollama: 노트북에서 로컬 LLM을 실행하고 API로 호출할 수 있게 해주는 실행 프로그램
- ✓ Qwen3:8B: 자연어 요청을 읽고 설계 내용을 해석하는 실제 LLM

### JSON 파일의 내부 형상

- ✓ STEP 5 기준 예시 (맨 아래는 실제 JSON 파일 사진)

자연어 요청: Create one U-shaped shaft support bracket as a single solid in FreeCAD. Create a horizontal base plate with X length 100 mm, Y width 60 mm, and Z thickness 10 mm at origin X=0, Y=0, Z=0. Create a left vertical wall using a box with X thickness 10 mm, Y width 60 mm, and Z height 50 mm at X=0, Y=0, Z=10. Create a right vertical wall using a box with X thickness 10 mm, Y width 60 mm, and Z height 50 mm at X=90, Y=0, Z=10. Fuse the base plate and both walls into one solid. Create four cylindrical mounting holes through the base plate. Each mounting hole has diameter 6 mm, starts at Z=0, has height 10 mm, and points along positive Z. Place the hole centers at X=20,Y=15; X=80,Y=15; X=20,Y=45; and X=80,Y=45.



```
{
  "boxes": [
    {
      "id": "Box_1",
      "size": [
        100,
        60,
        10
      ],
      "position": [
        0,
        0,
        0
      ]
    },
    {
      "id": "Box_2",
      "size": [
        10,
        60,
        50
      ],
      "position": [
        0,
        0,
        10
      ]
    },
    {
      "id": "Box_3",
      "size": [
        10,
        60,
        50
      ],
      "position": [
        90,
        0,
        10
      ]
    }
  ],
  "holes": [
    {
      "id": "Hole_1",
      "diameter": 6,
      "length": 10,
      "position": [
        20,
        15,
        0
      ],
      "direction": [
        0,
        0,
        1
      ]
    },
    {
      "id": "Hole_2",
      "diameter": 6,
      "length": 10,
      "position": [
        80,
        15,
        0
      ],
      "direction": [
        0,
        0,
        1
      ]
    },
    {
      "id": "Hole_3",
      "diameter": 6,
      "length": 10,
      "position": [
        20,
        45,
        0
      ],
      "direction": [
        0,
        0,
        1
      ]
    },
    {
      "id": "Hole_4",
      "diameter": 6,
      "length": 10,
      "position": [
        80,
        45,
        0
      ],
      "direction": [
        0,
        0,
        1
      ]
    }
  ],
  "cylinders": null
}
```

## ■ 6. 첫 번째 큰 구조 변경: JSON 중간 모델 도입

### JSON 사용 이유?

- ✓ JSON은 Python 코드보다 검사하기 쉽다는 장점

(예) 구멍의 위치는 반드시 숫자 3개

```
"position": [20, 15, 0]
```

잘못된 경우

```
"position": [20, 15]
```

or

```
"position": "center"
```

⇒ 코드를 실행하기 전에 자연어를 어떻게 이해하고 계획을 세웠냐를 볼 수 있기 때문에 검사하기가 쉽다.

### JSON 도입의 효과

- ✓ LLM의 역할이 다음 처럼 축소

자연어를 읽는다

→ 형상을 분류한다

→ 치수와 위치를 JSON에 기록한다

- ✓ 아래와 같은 역할들은 이제 LLM이 아닌

- compile\_plan.ps1

- run\_freecad.ps1

와 같은 코드로 인해 실행된다.

- FreeCAD API 선택

- Python 변수 생성

- Boolean 코드 작성

- 문서 저장

- 결과 객체 생성

## 7. compile\_plan.ps1: 결정론적 컴파일러 도입

### 기존 문제

- ✓ LLM이 직접 코드를 생성하면 같은 요청이라도 달라질 수 있다.

(예시 1)

```
shape = box.fuse(cylinder).cut(hole)
```

(예시 2)

```
feature1 = doc.addObject(...)  
feature2 = doc.addObject(...)  
Part.cut(...)
```

(예시 3)

```
PartDesign::Feature
```

⇒ 즉, 결과 코드 구조를 예측할 수 없음

### 개선된 방식

- ✓ compile\_plan.ps1이 JSON 항목별로 정해진 Python 문장을 생성 [파일 내부]

박스: `$lines += "box_$i = Part.makeBox(...)"`

원통: `$lines += "cylinder_$i = Part.makeCylinder(...)"`

결합: `$lines += "shape = shape.fuse(...)"`

구멍: `$lines += "shape = shape.cut(...)"`

최종결과: `feature = doc.addObject("Part::Feature", "Result")  
feature.Shape = shape  
doc.recompute()`

[JSON]

```
{  
  "boxes": [...],  
  "cylinders": [...],  
  "holes": [...]  
}
```

[컴파일러 작동]

1. 모든 additive shape 생성
2. additive shape 전부 fuse
3. 모든 hole 생성
4. hole을 차례대로 cut
5. removeSplitter
6. 필렛/모따기
7. 최종 Result 생성

⇒ 즉, JSON만 같다면 항상 동일한 Free CAD Python 코드 생성

## 8. ask\_plan.ps1에서 요구사항 목록과 CAD 계획을 분리

### 문제 상황

- ✓ JSON을 도입한 뒤에도 새로운 문제가 발생
- ✓ STEP 5(U자형 브래킷)을 요청하던 중 문제 발생

(요구사항 목록)

- 바닥 박스 1개
- 왼쪽 벽 박스 1개
- 오른쪽 벽 박스 1개
- 바닥 구멍 4개
- 양쪽 벽의 축 구멍 2개

하지만, LLM이 일부 구멍을 빠뜨리거나, 직사각형 벽을 원통으로 분류해버리는 문제 발생.

⇒ JSON 형식은 맞아도 요청에 존재하는 물리적 형상 개수가 틀림

### 개선된 구조

- ✓ ask\_plan.ps1이 두 번의 해석을 수행하도록 바꿈

### [1단계]: 독립적인 요구사항 목록 생성

```
"features": [  
  {  
    "id": "base_plate",  
    "type": "box",  
    "description": "rectangular base plate"  
  },  
  {  
    "id": "left_wall",  
    "type": "box",  
    "description": "rectangular left wall"  
  },  
  {  
    "id": "mounting_hole_1",  
    "type": "hole",  
    "description": "first mounting hole"  
  }  
]
```

⇒ LLM에게 먼저 요청문에 포함된 물리적 형상을 하나씩 분리

## 8. ask\_plan.ps1에서 요구사항 목록과 CAD 계획을 분리

### 개선된 구조

[2단계]: 실제 CAD 계획 생성

```
{  
  "boxes": [...],  
  "cylinders": [...],  
  "holes": [...]  
}
```

⇒ 그 다음 별도로 치수·위치·방향을 포함한 plan.json을 생성

[3단계]: 두 결과 비교

(요구사항 목록에서 계산한 개수 + 프롬프트 자체에서 보임)

```
Required boxes: 3  
Required solid cylinders: 0  
Required holes: 6
```

(아래 코드로 실제 계획의 배열 길이를 비교)

```
if ($boxes.Count -ne $ExpectedBoxes) {  
  $errors += "Expected $ExpectedBoxes boxes but found $($boxes.Count)."  
}
```

### 분리한 이유

✓ 하나의 LLM 호출에서 아래와 같은 두 일을 동시에 시키면  
모델이 스스로 만든 계획을 스스로 검증

- 무엇이 필요한지 판단
- 필요한 것을 실제 JSON으로 작성

(현재 구조) 아래와 같이 역할 분리

```
요구사항 목록 = 비교 기준  
CAD 계획 = 실제 실행 계획
```

## 9. 최대 3회 자동 수정 구조

### 문제 상황

- ✓ 베어링 지지대(STEP 7)를 요청했을 때  
요구사항 개수와 LLM 계획이 일치하지 않았다.

자연어 요청: Create one bearing support as a single solid in FreeCAD. Create a rectangular base plate with X length 120 mm, Y width 70 mm, and Z thickness 10 mm at X=0,Y=0,Z=0. Create a rectangular center pedestal with X length 60 mm, Y width 20 mm, and Z height 40 mm at X=30,Y=25,Z=10. Add one solid cylindrical bearing housing with diameter 50 mm and length 20 mm. Start the housing at X=60,Y=25,Z=50 and point it along positive Y. Fuse the base plate, center pedestal, and cylindrical housing. Create four cylindrical mounting holes through the base plate. Each mounting hole has diameter 8 mm, length 10 mm, and points along positive Z. Start the holes at X=15,Y=15,Z=0; X=105,Y=15,Z=0; X=15,Y=55,Z=0; and X=105,Y=55,Z=0. Create one cylindrical shaft hole through the bearing housing with diameter 20 mm and length 20 mm. Start it at X=60,Y=25,Z=50 and point it along positive Y. Subtract all five holes from the fused part.

- ✓ 직사각형 받침대도 박스여서 필요한 박스 개수가 더 많았고, 축 구멍을 포함하면 구멍 수도 달라짐.

```
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> powershell -ExecutionPolicy Bypass -File .\ask_plan.ps1
What should I create?: Create one bearing support as a single solid in FreeCAD. Create a rectangular base plate with X l
length 120 mm, Y width 70 mm, and Z thickness 10 mm at X=0,Y=0,Z=0. Create a rectangular center pedestal with X length 60
mm, Y width 20 mm, and Z height 40 mm at X=30,Y=25,Z=10. Add one solid cylindrical bearing housing with diameter 50 mm
and length 20 mm. Start the housing at X=60,Y=25,Z=50 and point it along positive Y. Fuse the base plate, center pedest
l, and cylindrical housing. Create four cylindrical mounting holes through the base plate. Each mounting hole has diamet
er 8 mm, length 10 mm, and points along positive Z. Start the holes at X=15,Y=15,Z=0; X=105,Y=15,Z=0; X=15,Y=55,Z=0; and
X=105,Y=55,Z=0. Create one cylindrical shaft hole through the bearing housing with diameter 20 mm and length 20 mm. Sta
rt it at X=60,Y=25,Z=50 and point it along positive Y. Subtract all five holes from the fused part.
Extracting independent feature checklist...
Corrected inconsistent hole_count using checklist length.
Corrected inconsistent cylinder_count using checklist length.
Required boxes: 1
Required solid cylinders: 2
Required holes: 4
Generating CAD plan...
Plan validation failed on attempt 1
- Expected 1 boxes but found 2.
PLAN_VALIDATION_SUCCESS
Saved requirements.json
Saved validated plan.json
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> |
```

- ✓ 또 다른 실행에서는 직사각형 받침대를 원통으로 잘못 분류

```
Plan validation failed on attempt 1
- Cylinder 'rectangular_pedestal' has an invalid diameter.
- Cylinder 'rectangular_pedestal' has an invalid length.
- Cylinder 'rectangular_pedestal' has unsupported direction 0,0,0.
- Duplicate id 'rectangular_pedestal'.
Plan validation failed on attempt 2
- Cylinder 'rectangular_pedestal' has an invalid diameter.
- Cylinder 'rectangular_pedestal' has an invalid length.
- Cylinder 'rectangular_pedestal' has unsupported direction 0,0,0.
- Duplicate id 'rectangular_pedestal'.
Plan validation failed on attempt 3
- Cylinder 'rectangular_pedestal' has an invalid diameter.
- Cylinder 'rectangular_pedestal' has an invalid length.
- Cylinder 'rectangular_pedestal' has unsupported direction 0,0,0.
- Duplicate id 'rectangular_pedestal'.
PLAN_VALIDATION_FAILED
```

## 9. 최대 3회 자동 수정 구조

### 자동 수정 방식

- ✓ 계획이 검증에 실패하면 오류 목록과 잘못된 계획을 LLM에 다시 전달

Current invalid plan:

{...}

Validation errors:

- Expected 3 boxes but found 2.
- Cylinder 'rectangular\_pedestal' has an invalid diameter.
- Duplicate id 'rectangular\_pedestal'.

Return a complete replacement plan.

⇒ 해당 과정을 최대 3회 반복하도록 함.

```
$maxRepairs = 3

for ($attempt = 1; $attempt -le $maxRepairs; $attempt++) {
    $errors = Test-PlanSchema ...

    if ($errors.Count -eq 0) {
        break
    }

    $plan = Invoke-JsonModel -Prompt $repairPrompt
}
```

### 성공 결과

- ✓ 왜 3회인가?

⇒ 실험하기 좋은 반복 수로, 임의로 설정함.

- ✓ 결과

Required boxes: 2  
Required solid cylinders: 1  
Required holes: 5  
PLAN\_VALIDATION\_SUCCESS

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> powershell -ExecutionPolicy Bypass -File sk_plan.ps1
What should I create?: Create one bearing support as a single solid in FreeCAD. Create a rectangular base plate with length 120 mm, Y width 70 mm, and Z thickness 10 mm at X=0,Y=0,Z=0. Create a rectangular center pedestal with X length 60 mm, Y width 20 mm, and Z height 40 mm at X=30,Y=25,Z=10. Add one solid cylindrical bearing housing with diameter 56 mm and length 20 mm. Start the housing at X=60,Y=25,Z=50 and point it along positive Y. Fuse the base plate, center pedestal, and cylindrical housing. Create four cylindrical mounting holes through the base plate. Each mounting hole has diameter 8 mm, length 10 mm, and points along positive Z. Start the holes at X=15,Y=15,Z=0; X=105,Y=15,Z=0; X=15,Y=55,Z=0; X=105,Y=55,Z=0. Create one cylindrical shaft hole through the bearing housing with diameter 20 mm and length 20 mm. Start it at X=60,Y=25,Z=50 and point it along positive Y. Subtract all five holes from the fused part.
Extracting independent feature inventory...
Required boxes: 2
Required solid cylinders: 1
Required holes: 5
Generating CAD plan...
PLAN_VALIDATION_SUCCESS
Saved requirements.json
Saved validated plan.json
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> |
```

⇒ 분류 규칙을 개선한 후 요구사항과 계획이 일치

## 10. Qwen3:8B 도입과 모델 역할 분리

### 왜 Qwen3:8B를 사용했나

- ✓ 기존 Qwen2.5-Coder:7B는 Python 코드를 직접 생성하는 데 사용
- ✓ 하지만 구조를 바꾼 뒤에 Qwen의 핵심 역할은 아래와 같이 변경

- 자연어 해석
- 형상 분류
- 구조화된 JSON 출력
- 검증 오류를 읽고 계획 수정

⇒ 따라서 더 높은 버전의 Qwen3:8B를 설치해 계획 생성 모델로 사용

- ✓ 하지만, 높은 버전을 사용했다고 오류가 자동으로 해결되지는 않았다.

⇒ 결국, 모델 교체보다 검증기와 결정론적 처리 구조 더 중요했다

### 최종 역할 분리

- ✓ Qwen3:8B

자연어 → 제한된 JSON 설계 계획

담당:

- 형상의 종류 파악
- 사용자의 설계 의도 해석
- 치수·위치·방향 구조화

- ✓ compile\_plan.ps1

검증된 JSON → 정확한 FreeCAD Python

- ✓ FreeCAD

Python → 실제 B-Rep CAD 형상

+) Boolean 연산이란?

두 개 이상의 3D 형상을 집합(set) 연산처럼 조합해서 새로운 형상을 만드는 방법

- ✓ 이 구조의 장점

⇒ 동일한 plan.json이면 항상 동일한 generated.py가 생성돼

[compile]

[FreeCAD]

담당:

- Part.makeBox
- Part.makeCylinder
- fuse
- cut
- makeFillet
- makeChamfer
- Profile 생성
- NACA 공식 코드 생성
- 최종 Result 객체 생성

담당:

- 기하 계산
- Boolean 연산
- Wire/Face/Solid 생성
- FCStd 저장
- STEP/STP 내보내기

LLM 출력의 변동성  
→ JSON 단계까지만 허용  
기하 계산과 CAD 코드  
→ 결정론적으로 고정

## 스키마 검증이란?

### 스키마

- ✓ 별도의 파일이나 프로그램이 아닌 ask\_plan 안에 있는 함수이다.
- ✓ ask\_plan.ps1이 자기 안에서 자기가 만든 plan을 스스로 검사하는 함수 호출. Compile\_plan.ps1이나 run\_freecad.ps1 같은 다른 파일에는 이 검증 로직이 없음.
- ✓ 함수가 하는 일
  - 개수 확인(요구사항과 실제 plan의 박스/구멍/실린더 개수가 맞는지)

```
if ($boxes.Count -ne $ExpectedBoxes) {  
    $errors += "Expected $ExpectedBoxes boxes but found $($boxes.Count)."  
}
```

- 각 항목별 필드 확인 (size 3개 있는지, diameter가 양수인지 등)

```
foreach ($box in $boxes) {  
    $ids += [string]$box.id  
    if (-not (Test-PlanVector3 $box.size)) {
```

- 허용 안 된 필드가 섞여 있는지 확인  
(예: box에 center 필드가 있으면 오류)

```
}  
  
return @($errors)  
}
```

### 내부 스키마 구현 사진

```
function Test-PlanSchema {  
    param(  
        $Plan,  
        [int]$ExpectedBoxes,  
        [int]$ExpectedCylinders,  
        [int]$ExpectedHoles,  
        [int]$ExpectedProfilePrisms,  
        [int]$ExpectedNacaWings,  
        [int]$ExpectedNacaLoftWings  
    )  
}
```

```
$maxRepairs = 3  
for ($attempt = 1; $attempt -le $maxRepairs; $attempt++) {  
    $errors = @(Test-PlanSchema -Plan $plan ...) # + 검증 실행  
  
    if ($errors.Count -eq 0) {  
        break # 통과하면 루프 탈출  
    }  
    # 실패하면 오류 메시지를 담아 LLM에게 재수정 요청  
    ...  
}  
  
if ($finalErrors.Count -gt 0) {  
    exit 1 # 4번 다 실패하면 plan.json을 아예 저장하지 않고 종료  
}  
  
$plan | ConvertTo-Json | Set-Content ".\plan.json" # 검증 통과한 것만 저장
```

### 스키마 구조 실행

### 구조 실행 (코드)

: ask\_plan.ps1 안에서, plan.json으로 저장되기 직전에 최대 4번 (최초 1번 + 재시도 3번) 일어남

```
① LLM이 plan 생성 (Invoke-JsonModel)  
↓  
② 결정론적 후처리 (id 재부여, center→position 별칭 수정 등)  
↓  
③ Test-PlanSchema 호출 ← 여기가 검증 시점  
↓  
통과? —예→ 다음 단계로 진행 (compile_plan.ps1이 쓸 수 있는 plan.json 저장)  
|  
아니오  
↓  
오류 메시지와 함께 LLM에게 재수정 요청 → ②로 돌아가서 반복 (최대 3회)
```

## 11. 스키마 검증기의 발전

### 초기 검증

✓ 처음에는 배열 개수와 기본 필드만 검사했다.

- 박스 개수
- 원통 개수
- 구멍 개수
- `size` 길이
- `position` 길이
- `direction` 길이
- 양수 치수

### 추가된 검증

✓ 실험 중 새로운 오류가 나타날 때마다 다음 검증을 추가

#### [ID 검증]

- ID가 비어 있지 않은가?
- ID가 중복되지 않는가?
- `Result` 를 중간 형상 ID로 사용하지 않았는가?

이유: LLM이 박스 ID를 Result, 필렛 ID도 Result로 생성한 적이 있었기 때문에  
Result는 최종 FreeCAD 객체에만 사용하도록 예약

## 11. 스키마 검증기의 발전

### 최상위 속성 제한

- ✓ 허용된 JSON 최상위 항목만 통과

```
boxes
cylinders
holes
profile_prisms
naca_wings
naca_loft_wings
fillets
chamfers
```

- ✓ 아래와 같은 항목들은 컴파일러가 아직 해석하지 못하므로 LLM이 이와 같은 항목을 임의로 생성해도 통과하지 못하도록 제한함.

```
constraints
dimensions
notes
operations
features
parameters
metadata
geometry
```

### 빈 배열과 누락 항목 문제

- ✓ LLM이 요청하지 않은 형상을 다음처럼 빈 객체로 만들기도 함

```
json
{
  "boxes": [{}],
  "holes": [{}],
  "cylinders": [{}]
```

이렇게 되버리면, 개수는 1로 계산되지만 실제 정보가 없음

⇒ 따라서 각 항목 내부의 필수 필드와 배열 길이도 검사하게 함

## 12. 결정론적 ID 부여

### 문제 상황과 변경

✓ LLM이 다음과 같은 문제를 반복함.

- ID 누락
- 동일 ID 재사용
- `Result` 를 중간 ID로 사용
- 복구 시 ID가 변경됨

✓ 변경 내용

```
Box_1
Box_2
Cylinder_1
Hole_1
ProfilePrism_1
Fillet_1
Chamfer_1
NacaWing_1
```

⇒ LLM의 ID를 전적으로 신뢰하지 않고, 계획을 받은 후 배열 순서에 따라 ID를 다시 부여하는 구조로 변경

### 작동 방식과 효과

✓ 작동방식 (ask\_plan 함수 Set-DeterministicPlanIds 내부에 존재)

```
$index = 1

foreach ($item in @($Plan.bboxes)) {
    if ($null -ne $item) {
        $item.id = "Box_$index"
        $index++
    }
}
```

✓ 효과

- ID 중복 제거
- 복구 시 ID 안정화
- `Result` 예약어 충돌 방지
- LLM이 이름을 잘못 생성해도 형상 계획은 유지

## 13. 실제 CAD 형상 검증 추가

### 문제 상황

- ✓ FreeCAD 화면에 형상이 보인다고 해서 올바른 결과는 아니었다.

#### [가능한 오류]

- 여러 Solid가 서로 떨어져 있음
- Boolean 연산 실패
- 내부적으로 유효하지 않은 Shape
- 구멍이 실제로 관통하지 않음
- 요청 치수와 Bounding Box가 다름
- 축 방향이 틀림
- 부피가 예상과 다름

### 검증 항목 추가

- ✓ run\_freecad.ps1 파일에서 다음을 검사하도록 명령

#### [내부 코드]

```
obj.Shape.BoundingBox.XLength  
obj.Shape.BoundingBox.YLength  
obj.Shape.BoundingBox.ZLength  
len(obj.Shape.Solids)  
obj.Shape.Volume  
obj.Shape.IsValid()
```

#### [출력 예]

```
Solid=1  
AND isValid=True  
AND 주요 치수 일치
```

```
BOUNDING_BOX=80.000000,50.000000,15.000000  
SOLID_COUNT=1  
VOLUME=26314.601234  
SHAPE_VALID=True  
FREECAD_SUCCESS
```

#### [효과]

⇒ run\_freecad.ps1 코드를 마지막에 Powershell에 실행시켰을 때,  
출력 예시와 같이 뜬다면 검증 성공

## 14. 필렛 기능 추가와 Edge 번호 문제

### 문제 상황

✓ 자연어 요청: STEP 8 - 직육면체의 Z축 방향 모서리에 반지름 5 mm 필렛을 적용

✓ 문제 - 토폴로지 명칭 안전성 문제

: FreeCAD의 Edge1, Edge2 같은 번호는 Boolean 연산이나 형상 변화 후 바뀔 수 있다.

(예)

```
shape.makeFillet(5, [shape.Edges[0], shape.Edges[4]])
```

는 단순 형상에서는 작동하지만, 앞에 구멍 하나가 추가 되면

이후에 다른 Edge를 가리킬 수 있다.

❖ 토폴로지 (topology)란?

⇒ 3D 형상을 이루는 점·선·면·덩어리 사이의 연결 구조.

"이 도형이 정확히 몇 mm냐"와는 다른 개념이고,

"이 면이 어떤 엣지들로 둘러싸여 있고, 그 엣지가 어떤 정점 2개를 잇느냐" 같은 구조·관계

### 개선

✓ Edge 번호 대신 기하학적 방향을 검사

(예) 해당 코드는 컴파일러 파일에 존재

```
for edge in shape.Edges:
    if isinstance(edge.Curve, Part.Line):
        edge_direction = edge.Vertexes[-1].Point.sub(
            edge.Vertexes[0].Point
        )

        if edge_direction.cross(fillet_axis).Length <= tolerance:
            fillet_edges.append(edge)
```

[결과]

“Edge 3을 필렛”

→ 불안정

“Z축과 평행한 모든 직선 Edge를 필렛”

→ 상대적으로 안정적

## 추가 문제 상황

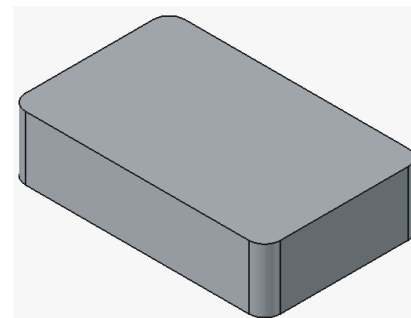
- ```
$lines += "if not fillet edges $i:"
```

- ✓ PowerShell이 \$i:를 하나의 변수 이름처럼 해석해서 ParserError가 발생

## 개선&결과

- ```
$lines += "if not fillet edges ${i}:"
```

⇒ Z축과 평행한 네 모서리에 필렛이 적용된 블록 생성에 성공



## 15. 모따기 기능 추가

### 추가 과정

- ✓ 필렛과 동일한 축 방향 선택 방식을 모따기 에도 확장

[ask\_plan 파일에 추가]

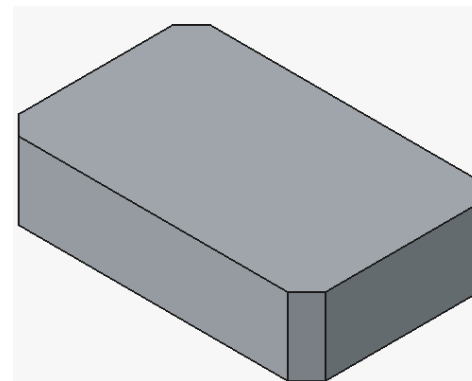
```
{
  "chamfers": [
    {
      "id": "Chamfer_1",
      "distance": 5,
      "parallel_to": [0, 0, 1]
    }
  ]
}
```

[컴파일러 파일에 추가]

```
shape = shape.makeChamfer(distance, chamfer_edges)
```

### 추가 검증과 결과

- ✓ 추가 검증을 거침
  - `distance > 0`
  - `parallel_to` 숫자 3개
  - 허용 축인지 확인
  - 빈 chamfer 배열 지원
  - 필렛과 ID가 중복되지 않는지 확인
  - `chamfers` 를 허용된 최상위 속성에 등록
- ✓ 결과 (모따기에도 동일하게 적용 완료)



## 16. 임의 단면을 위한 profile\_prisms 추가

### 추가 과정

- ✓ 날개 형상을 만들기 위한 첫번째 과정 시작  
그러기 위해서 점 목록을 받아 Face를 만든 뒤 압출하는 기능 추가

[ask\_plan 파일에 추가]

[컴파일러 파일에 추가]

```
{
  "profile_prisms": [
    {
      "id": "ProfilePrism_1",
      "curve_type": "polyline",
      "points": [
        [0, 0, 0],
        [40, 0, 0],
        [20, 30, 0]
      ],
      "extrusion": [0, 0, 10]
    }
  ]
}
```

```
profile_points = [
  App.Vector(0, 0, 0),
  App.Vector(40, 0, 0),
  App.Vector(20, 30, 0)
]

profile_wire = Part.makePolygon(
  profile_points + [profile_points[0]]
)

profile_face = Part.Face(profile_wire)
profile_prism = profile_face.extrude(App.Vector(0, 0, 10))
```

### 문제 상황과 해결

- ✓ 우리가 요구한 내용

[0, 0, 0]

하지만, 점을 배열이 아니라 객체로 반환

⇒ {"x": 0, "y": 0, "z": 0}

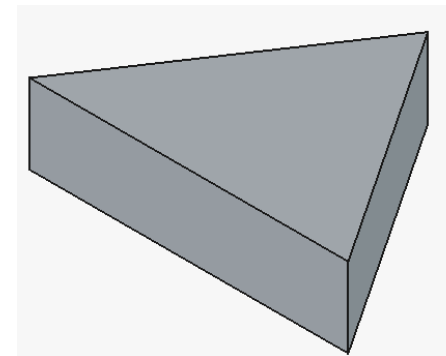
- ✓ 개선: 프롬프트와 스키마(ask\_plan 파일)에 다음 규칙을 추가

Every profile point must be a JSON array of exactly three numbers.  
The extrusion must be a JSON array of exactly three numbers.  
The top-level object must contain only the allowed schema.

- ✓ 결과

[스키마 검증 성공]

Bounding Box: 40 x 30 x 10  
Volume: 6000  
Solids: 1  
isValid: True



## 17. 다각형 단면에서 B-spline 단면으로 발전

### 문제 상황과 해결

#### ✓ 문제 상황

: 다각형 점들을 직선으로 연결하면 날개처럼 보이지만 실제 표면이 각져있다.

하지만, 에어포일은 부드러운 곡선이 필요

#### ✓ 해결 과정

ask\_plan 파일 내부의 planPrompt 안의 스키마 예시 템플릿 부분에 curve\_type이라는 두가지 명령을 추가 해줌

```
"curve_type": "polyline"
```

```
"curve_type": "periodic_bspline"
```

❖ B-spline: 점들을 지나는 부드러운 곡선

### 작동 방식과 효과 + 한계

#### ✓ 작동 방식

[아래는 앞 전 코드를 추가 함으로써 컴파일 결과 사진]

##### - Polyline

```
profile_wire = Part.makePolygon(  
    points + [points[0]]  
)
```

##### - Periodic B-spline

```
curve = Part.BSplineCurve()  
curve.interpolate(  
    Points=points,  
    PeriodicFlag=True  
)  
wire = Part.Wire(curve.toShape())
```

#### ✓ 효과

- 단면 곡선이 부드러워짐
- 자유곡선 형상 가능
- 날개 단면 실험 가능

✓ 한계: B-spline 보간곡선은 입력 점을 통과하지만, NACA 공식 형상을 정확히 보장하지 않음, 또한 주기적 B-spline은 곡선 특성 때문에 입력 점의 최소·최대 범위를 약간 벗어날 수 있다.

실제 실험에서도 수치적인 부분이 조금 오류가 발생했다.

## 18. NACA 좌표를 LLM이 만들지 않도록 변경

### 문제 상황

✓ LLM에게 날개 모양 점들을 직접 생성하라고 할 때 문제점 발생

- 매번 좌표가 달라짐
- 일부 좌표 누락 가능
- 점 순서가 뒤바뀔 수 있음
- NACA 0012라고 해도 실제 NACA 공식과 다를 수 있음
- 샘플 수가 많을수록 출력 오류 가능성 증가
- 좌표 반올림 때문에 두께와 시위가 변함
- 정확한 재현과 검증이 어려움

### 새로운 역할 분리

✓ LLM은 아래와 같이 설계값만 작성

```
{  
  "designation": "0012",  
  "chord": 100,  
  "span": 300,  
  "samples": 101,  
  "trailing_edge": "closed",  
  "position": [0, 0, 0]  
}
```

✓ 컴파일러가 아래 NACA 공식을 이용해 좌표를 계산해서 python 코드 작성

$$y_t = 5t( \text{coefficient} \cdot x^4 - 0.2969\sqrt{x} - 0.1260x - 0.3516x^2 + 0.2843x^3 )$$

## 18. NACA 좌표를 LLM이 만들지 않도록 변경

### 계산 순서

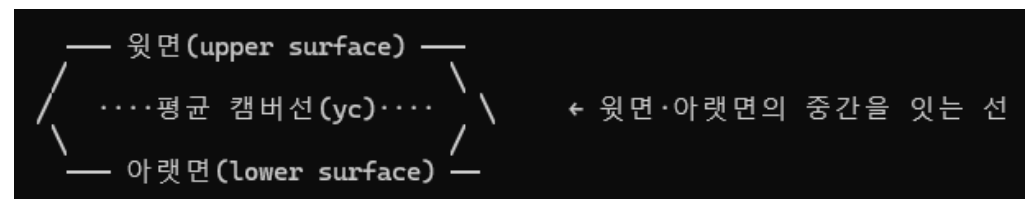
- ✓ 컴파일러(maka\_naca\_section 함수) 코드 실행 시 이와 같은 python 코드 생성

```
for sample_index in range(samples):  
    x = 0.5 * (1.0 - math.cos(math.pi * sample_index / (samples - 1))) # ②x 좌표(0~1) 생성  
    yt = 5.0 * t * (0.2969*x - 0.1260*x - 0.3516*x2 + 0.2843*x3 + coeff*x4) # ③두께 계산  
    ...  
    yc = ... # ④캠버선(중심선) 높이 계산  
    theta = ... # ⑤캠버선의 기울기(각도)  
    xu = x - yt * sin(theta) # ⑥윗면 X좌표 = 중심선에서 두께만큼 수직으로 밀어냄  
    zu = yc + yt * cos(theta) # ⑥윗면 Z좌표  
    xl = x + yt * sin(theta) # ⑦아랫면 X좌표  
    zl = yc - yt * cos(theta) # ⑦아랫면 Z좌표  
    upper.append(App.Vector(...xu..., ...zu...)) # ⑧실제 3D 좌표점으로 저장  
    lower.append(App.Vector(...xl..., ...zl...))
```

- Yt 자체는 좌표가 아닌 해당 x 위치에서 날개가 얼마나 두꺼운가에 대한 값
- X와 yt 값들이 적용된 (xu, zu, 티, 키) 값 들이 실제 3D 좌표이다  
⇒ 이렇게 만들어진 좌표들이 컴파일러 내부 함수 make\_naca\_section() 안에 Part.BSplineCurve().interpolate(Points=upper) 를 통해 곡선 → 면 → 압출 순서를 거쳐 3D 솔리드가 된
- ❖ Part.BSplineCurve().interpolate(Points=upper) 뜻  
: Part.BSplineCurve()로 빈 곡선 틀을 하나 만들고, .interpolate(Points=upper)는 이 점들을 순서대로 정확히 지나가는 부드러운 곡선을 계산해서 채워 넣으라는 코드

### 평균 캠버선 & 장점

- ✓ 평균 캠버선: 날개 단면(에어포일)의 윗면과 아랫면의 정중앙을 잇는 가상의 곡선



m = 최대 캠버  
p = 최대 캠버 위치  
t = 최대 두께

를 추출해서 상·하부 좌표를 계산

- ✓ 장점: 동일한 입력이면 항상 동일한 좌표가 도출

```
NACA 0012  
chord 100  
samples 101  
closed trailing edge
```

⇒ 위 내용이 같으면 결과 좌표도 같음.

따라서, LLM은 설계 의도를 담당하고, 수학적으로 정확한 좌표는 컴파일러를 통해 Python이 담당

## 19. NACA 직선 압출 날개

### 첫 NACA 전용 형식

- ✓ 첫 NACA 전용 형식은 일정한 단면을 스캔 방향으로 압출하는 방식 사용

```
{
  "naca_wings": [
    {
      "designation": "0012",
      "chord": 100,
      "span": 300,
      "samples": 101,
      "trailing_edge": "closed",
      "position": [0, 0, 0]
    }
  ]
}
```

해당 코드는 ask\_plan 파일 내부  
\$planPrompt 안의  
(LLM에게 보내는 프롬프트)  
naca\_wing 스키마 예시 템플릿에 위치

- ✓ Python 작동 방식

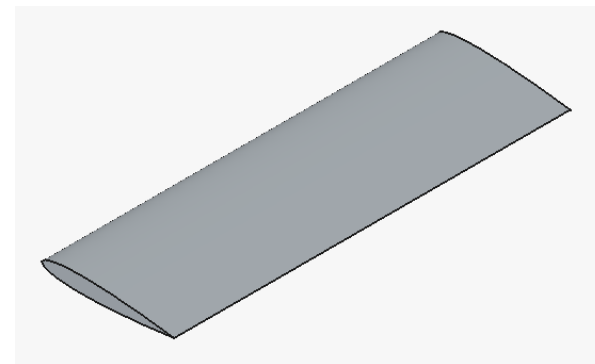
1. NACA 단면 좌표 계산
2. 상부 B-spline 생성
3. 하부 B-spline 생성
4. 폐곡선 Wire 생성
5. Face 생성
6. Y축 방향으로 압출

### 결과

- ✓ 자연어 요청 사항: Create one NACA 0012 wing as a single solid in FreeCAD. Use the four-digit designation 0012, chord length 100 mm, span 300 mm, 101 samples per surface, a closed trailing edge, and position [0,0,0]. Generate the airfoil coordinates deterministically from the NACA four-digit equations. Do not create any boxes, cylinders, holes, profile prisms, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.

- ✓ 결과 값과 실제 사진

- X 길이: 100 mm
- Y 길이: 300 mm
- Z 두께: 약 11.994 mm
- Solid: 1
- isValid: True



## 20. 로프트 날개로 확장

### 추가된 설계값

- ✓ 이전 과정에서 압축은 루트와 팁 단면이 같다.
- ✓ 실제 날개처럼 만들기 위해 두 단면을 별도로 생성하고 그 사이를 자연스럽게 연결하도록 하는 방식.

[ask.plan 파일 내부 스키마 템플릿에 새로 추가]

```
{
  "designation": "0012",
  "root_chord": 120,
  "tip_chord": 60,
  "span": 300,
  "sweep_offset": 40,
  "tip_z_offset": 20,
  "samples": 101,
  "trailing_edge": "closed",
  "position": [0, 0, 0]
}
```

### 작동 방식 & 결과

#### ✓ 작동 방식

##### [루트 단면]

위치: [0, 0, 0]  
시위: 120

##### [팁 단면]

위치: [40, 300, 20]  
시위: 60

설정 한 뒤에 두 단면을 이어붙이라는 컴파일러 내부 명령 실행

```
shape = Part.makeLoft(
    [root_wire, tip_wire],
    True,
    False
)
```

여기서:

- 첫 번째 `True` : Solid 생성
- 두 번째 `False` : ruled surface가 아닌 부드러운 loft

#### ✓ 결과

⇒

- 테이퍼 적용
- 스윙 적용
- 팁 높이 변화 적용
- 단일 Solid 생성

## 21. 가장 중요한 치수 문제: tip\_chord 60 → 6

### 문제 상황

- ✓ 자연어 요청 사항 `tip chord 60 mm`

하지만, LLM이 JSON에서 다음처럼 출력

```
"tip_chord": 6
```

- ✓ 6은 (숫자, 양수, 필수 항목, JSON 문법 정상)이므로 일반 스키마 검증은 통과한다.
- ✓ 문제 상황에 대한 프롬프트 출력

```
Corrected tip_chord from 6 to 60 using the original request.  
PLAN_VALIDATION_SUCCESS
```

```
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작> powershell -ExecutionPolicy Bypass -File .\ask_plan.ps1  
What should I create?: Create one twisted tapered NACA 0012 loft wing as a single solid in FreeCAD. Use NACA designation 0012, root chord 120 mm, tip chord 60 mm, span 300 mm, sweep offset 40 mm, tip Z offset 20 mm, tip twist angle -3 degrees, twist axis fraction 0.25 measured from the tip leading edge, 101 samples per section, a closed trailing edge, and position [0,0,0]. Generate the root and tip airfoil sections deterministically from the NACA four-digit equations, rotate the tip section using the right-hand rule about an axis parallel to positive Y, and connect the sections using a solid loft. Do not create any boxes, cylinders, holes, straight-extruded NACA wings, profile prisms, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.  
Extracting independent feature inventory...  
Required boxes: 0  
Required solid cylinders: 0  
Required holes: 0  
Required profile prisms: 0  
Required NACA wings: 0  
Required NACA loft wings: 1  
Generating CAD plan...  
Corrected tip_chord from 6 to 60 using the original request.  
PLAN_VALIDATION_SUCCESS  
Saved requirements.json  
Saved validated plan.json  
PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작>
```

### 출력의 의미와 한계

- ✓ 오른쪽 사진의 출력이 의미하는 것은?

⇒ LLM이 60을 잘못 써서 6으로 고쳤다는 문구

- ✓ 한계

우리가 설정한 스키마 검증은 다음 질문에 답한다.

```
tip_chord가 존재하는가?  
숫자인가?  
양수인가?
```

하지만, 결론적으로 스키마는 다음 질문에는 답을 하지 못 함.

```
사용자가 정말 요청한 값과 같은가?
```

## 21. 가장 중요한 치수 문제: tip\_chord 60 → 6

### 최종적인 새로운 방식

- ✓ 스키마 검증을 통해 오류를 잡아냈지만, 더욱 완벽한 프로그램을 위해 최종적인 방식 도출

```
① 사용자 원문 치수
② → LLM이 JSON 계획 생성
③ → 원문에서 핵심 숫자를 규칙 기반으로 재추출
④ → JSON 값과 비교
⑤ → 다르면 원문 값으로 교정
⑥ → 다시 스키마 검증
⑦ → 통과 후 컴파일
```

[핵심]

- ③ 원문에서 핵심 숫자를 규칙 기반으로 재추출

```
root_chord = Get-RequiredRequestNumber `
    -Request $Request `
    -Pattern "\broot\s+chord(?:\s+length)?\s*(?:=|:|of)?\s*$number" `
    -FieldName "root_chord"
```

⇒ ask\_plan 파일 내부에 해당 코드를 통해 원문 텍스트를  
직접 뒤져보고 특정 숫자를 뽑아냄.

- LLM의 판단이 아니라 고정된 문자열 규칙으로 숫자를 찾는 방식

### ④ JSON 값과 비교

```
$oldJson = $property.Value | ConvertTo-Json -Compress # LLM이 쓴 값: "6"  
$newJson = $entry.Value | ConvertTo-Json -Compress # 원문에서 뽑은 값: "60"
```

⇒ 앞 슬라이드에서 나온 출력 메시지도 해당 과정을 거쳐  
다름을 확인한 뒤, 고쳤다는 메시지를 받았음.

### ⑥ 다시 스키마 검증

```
$errors = @(Test-PlanSchema -Plan $plan -ExpectedBoxes ... -ExpectedNacaLoftWings $expectedNacaLoftWings)
```

⇒ 마지막으로 스키마 검증을 통해 한번 더 검사 과정을 거친 뒤,  
통과하면, 그때 컴파일러를 실행한다.

## 21. 가장 중요한 치수 문제: tip\_chord 60 → 6

### 결정론적 재추출 예

✓ PowerShell 정규표현식으로 다음 항목을 원문에서 특정 수치를 찾는다

- designation
- root chord
- tip chord
- span
- sweep offset
- tip Z offset
- tip twist
- twist axis fraction
- samples
- trailing edge
- position

### 재추출 예시와 연구적 의의

✓ 재추출 예시 코드

```
$tipChordMatch = [regex]::Match(  
    $Request,  
    'tip\s+chord\s+(-?\d+(?:\.\d+)?)',  
    IgnoreCase  
)
```

+

```
$exactValues.tip_chord = [double]$tipChordMatch.Groups[1].Value
```

⇒ 두 코드를 통해 LLM 계획과 비교해 다르면 교정한다

✓ 연구적 의의

⇒ 단순 스키마 검증보다 한 단계 높은 원문 기반 의미·치수 검증 과정 도달

## 22. 로프트 날개에 비틀림 추가 (마지막 STEP 15)

### 자연어 요청 사항

Create one twisted tapered NACA 0012 loft wing as a single solid in FreeCAD. Use NACA designation 0012, root chord 120 mm, tip chord 60 mm, span 300 mm, sweep offset 40 mm, tip Z offset 20 mm, tip twist angle -3 degrees, twist axis fraction 0.25 measured from the tip leading edge, 101 samples per section, a closed trailing edge, and position [0,0,0]. Generate the root and tip airfoil sections deterministically from the NACA four-digit equations, rotate the tip section using the right-hand rule about an axis parallel to positive Y, and connect the sections using a solid loft. Do not create any boxes, cylinders, holes, straight-extruded NACA wings, profile prisms, fillets, or chamfers. Add only the final shape to the document as one Part::Feature named Result.

### 추가 과정

- ✓ 비행기 날개 팁 단면에 회전을 적용함.
- ✓ Ask\_plan 파일 스키마 템플릿 naca\_loft\_wing에 해당 코드 추가

```
{  
  "tip_twist_deg": -3,  
  "twist_axis_fraction": 0.25  
}
```

- ✓ 컴파일러 파일에도 아래 코드 추가 후 회전 중심 계산

tip leading edge  
+ tip\_chord × 0.25

→  
추가 후 실험

tip leading edge X = 40  
tip chord = 60  
twist fraction = 0.25  
회전축 X = 40 + 60 × 0.25 = 55

⇒ 회전 중심 = (55, 300, 20)

## 22. 로프트 날개에 비틀림 추가

### 작동 방식

1. 회전되지 않은 팁 NACA 단면 생성
2. 회전축 원점 계산
3. Y축과 평행한 축 설정
4. 팁 Wire 회전
5. 루트 Wire와 회전된 팁 Wire를 Solid loft
6. `removeSplitter`
7. 최종 Solid 검증

```
tip_wire.rotate(  
    App.Vector(55, 300, 20),  
    App.Vector(0, 1, 0),  
    angle  
)
```

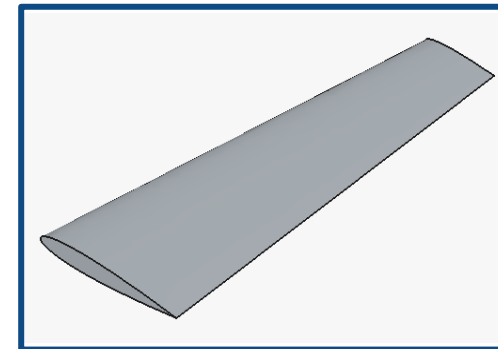
⇒ 이만큼 비틀라는 뜻

❖ `removeSplitter`: 로프트 날개 두개를 합칠 때 이음새를 지워주는 기능

### 결과

- ✓ 마지막 `run_freecad.ps1` 검증 결과

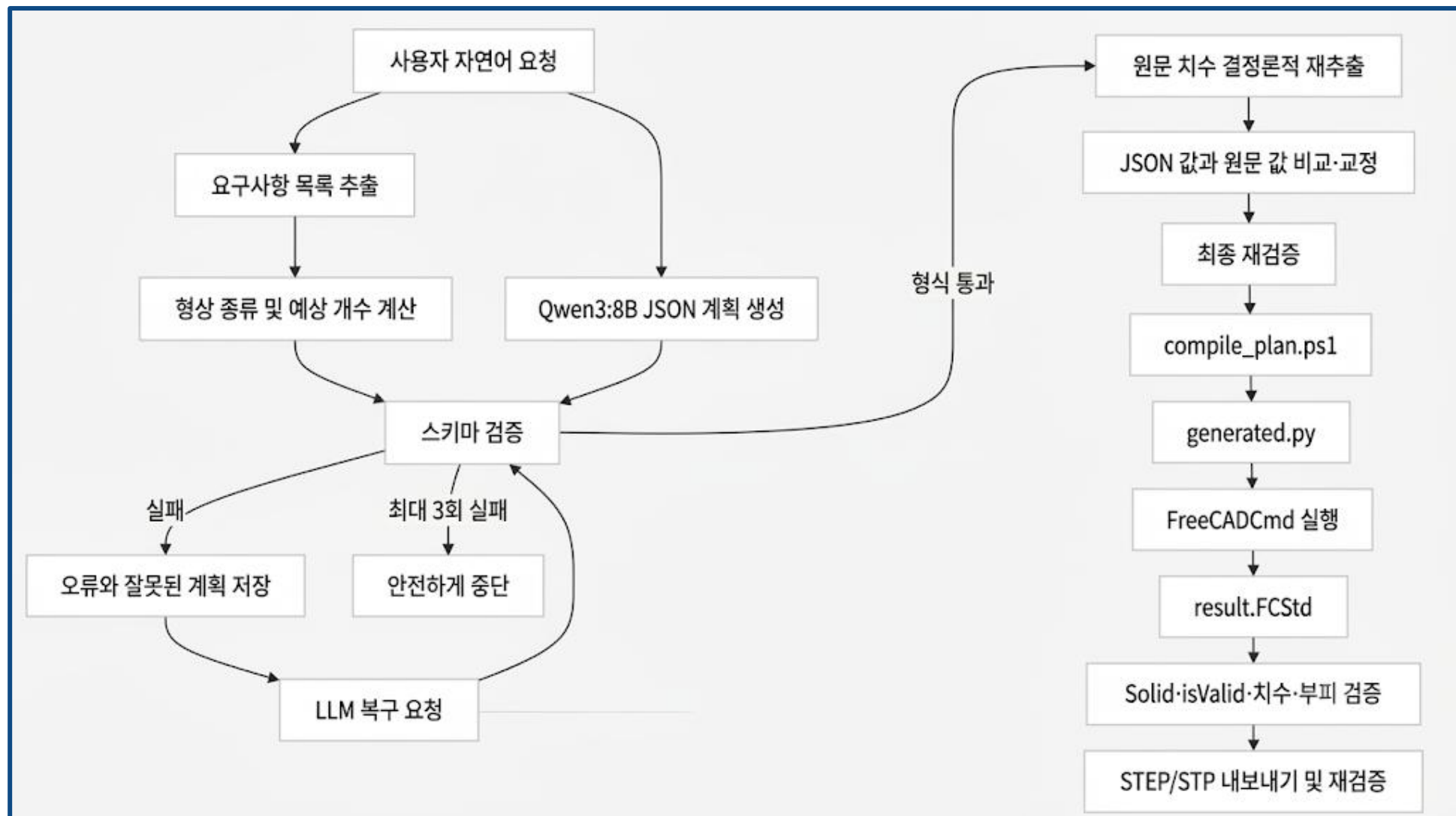
```
Solid count: 1  
isValid: True  
Volume: 약 205702.569856 mm3  
회전축: 약 (55, 300, 20)  
측정 각도: 약 3.000000°
```



- ✓ 유의할 점: 요청에서는  $-3^\circ$  였지만 FreeCAD 축 방향과 측정 순서에 따라 측정 부호가  $+3^\circ$ 로 보일 수 있다. 따라서 다음을 확인해야한다

- 회전축 방향
- 오른손 법칙
- 앞전에서 뒷전으로 잡은 벡터 방향
- 각도 측정 순서

## 23. 현재 프로그램의 최종 구조



## 24. 파일별 현재 역할

### ask\_plan.ps1

담당:

- 사용자 자연어 입력
- Ollama API 호출
- 요구사항 목록 생성
- 형상 종류 분류
- 필요 개수 계산
- LLM JSON 계획 생성
- 스키마 검증
- ID 안정화
- 오류 기록
- 최대 3회 자동 복구
- 원문 NACA 치수 재추출
- JSON 치수 비교·교정
- `requirements.json` 저장
- `plan.json` 저장

### requirements.json

역할:

- 요청에 포함된 형상의 독립적인 체크리스트
- 박스·원통·구멍·프로파일·NACA 날개 개수
- 실제 계획의 누락 여부를 검사하는 비교 기준

## ■ 24. 파일별 현재 역할

### raw\_plan\_initial.json

역할:

- LLM이 최초 생성한 원본 계획 저장
- 검증기나 교정기가 수정하기 전의 상태 기록
- 연구 실험에서 LLM 오류 분석에 사용

### invalid\_plan\_attempt\_N.json

역할:

- 각 복구 시도에서 실패한 계획 저장
- 어떤 오류가 반복됐는지 추적
- 프롬프트와 검증기 개선 근거 제공

## ■ 24. 파일별 현재 역할

### plan.json

역할:

- 검증과 교정이 끝난 최종 CAD 계획
- 컴파일러가 읽는 유일한 설계 입력

### compile\_plan.ps1

담당:

- JSON 읽기
- 형상별 FreeCAD Python 생성
- additive 형상 생성
- fuse
- hole cut
- 필렛
- 모따기
- Profile 압출
- NACA 공식 생성
- NACA loft 생성
- 팁 비틀림 적용
- 최종 `generated.py` 저장

## ■ 24. 파일별 현재 역할

### generated.py

역할:

- FreeCAD가 실제 실행하는 결정론적 Python 코드
- LLM이 직접 작성한 코드가 아님
- 동일한 `plan.json` 이면 동일하게 생성됨

### run\_freecad.ps1

담당:

- FreeCADCmd 실행
- `generated.py` 실행
- 최종 객체 검색
- Solid 개수 검사
- 형상 유효성 검사
- Bounding Box 및 부피 출력
- `result.FCStd` 저장
- FreeCAD GUI로 결과 열기

## 25. 초기 구조와 현재 구조의 본질적인 차이

### 초기

- ✓ LLM이 설계자 + 계산기 + 프로그래머 + CAD 실행 계획자 역할을 모두 수행
- ✓ 문제점
  - 환각
  - API 오류
  - 치수 누락
  - 매 실행 결과 변화
  - 복잡한 코드 생성
  - 실패 원인 분리가 어려움
  - 연구 결과의 재현성 부족

### 현재

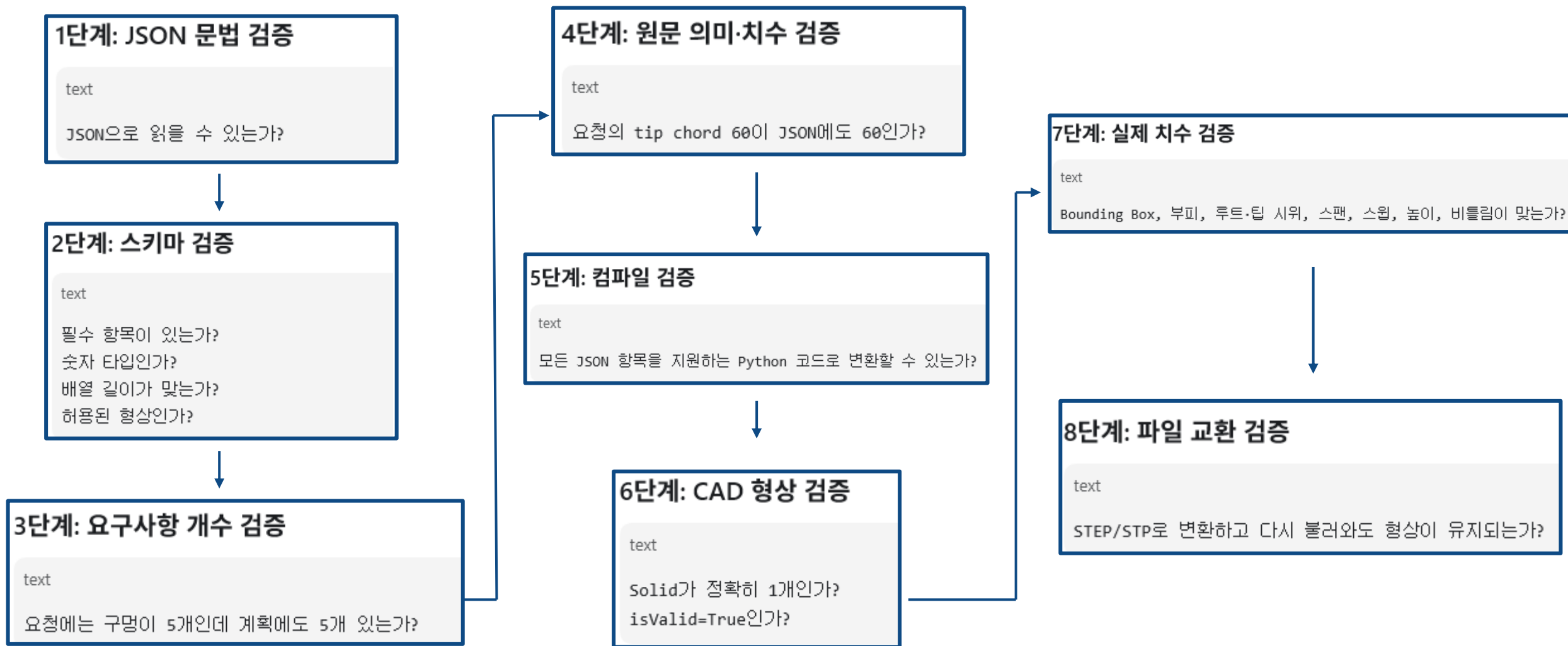
#### ✓ 각각의 역할

LLM = 자연어 해석기  
JSON = 제한된 설계 언어  
검증기 = 안전장치  
원문 비교기 = 치수 보존 장치  
컴파일러 = 정확한 코드 생성기  
Python = 수학 계산기  
FreeCAD = 기하 커널  
실제 형상 검증기 = 결과 판정기

#### ✓ 장점

- 어느 단계에서 문제가 발생했는지 구분 가능
- 잘못된 LLM 출력을 실행 전에 차단
- 동일 계획에서 동일 코드 생성
- NACA 좌표의 재현성 확보
- 실제 Solid와 치수까지 검증 가능
- 새로운 형상 타입을 모듈처럼 확장 가능

## 26. 연구 과정에서 형성된 다층 검증 구조



## ■ 27. 현재까지의 가장 중요한 연구적 결론

### 결론 1

LLM이 FreeCAD 코드를 직접 생성하는 것은 가능하지만, 범용적이고 정확한 시스템으로 만들기에는 변동성이 너무 크다.

### 결론 2

JSON을 사용하면 LLM의 자유도를 제한하고 검증 가능한 중간 표현을 만들 수 있다.

### 결론 3

요구사항 목록과 실행 계획을 분리하면 형상 누락을 더 잘 찾을 수 있다.

### 결론 4

스키마 검증은 구조적 오류를 잡지만, `60 → 6` 같은 의미상 치수 오류는 잡지 못했다.

### 결론 5

정확해야 하는 숫자는 원문에서 결정론적으로 다시 추출 또는 UI 입력·단위 파서·전용 문법으로 받는 형식

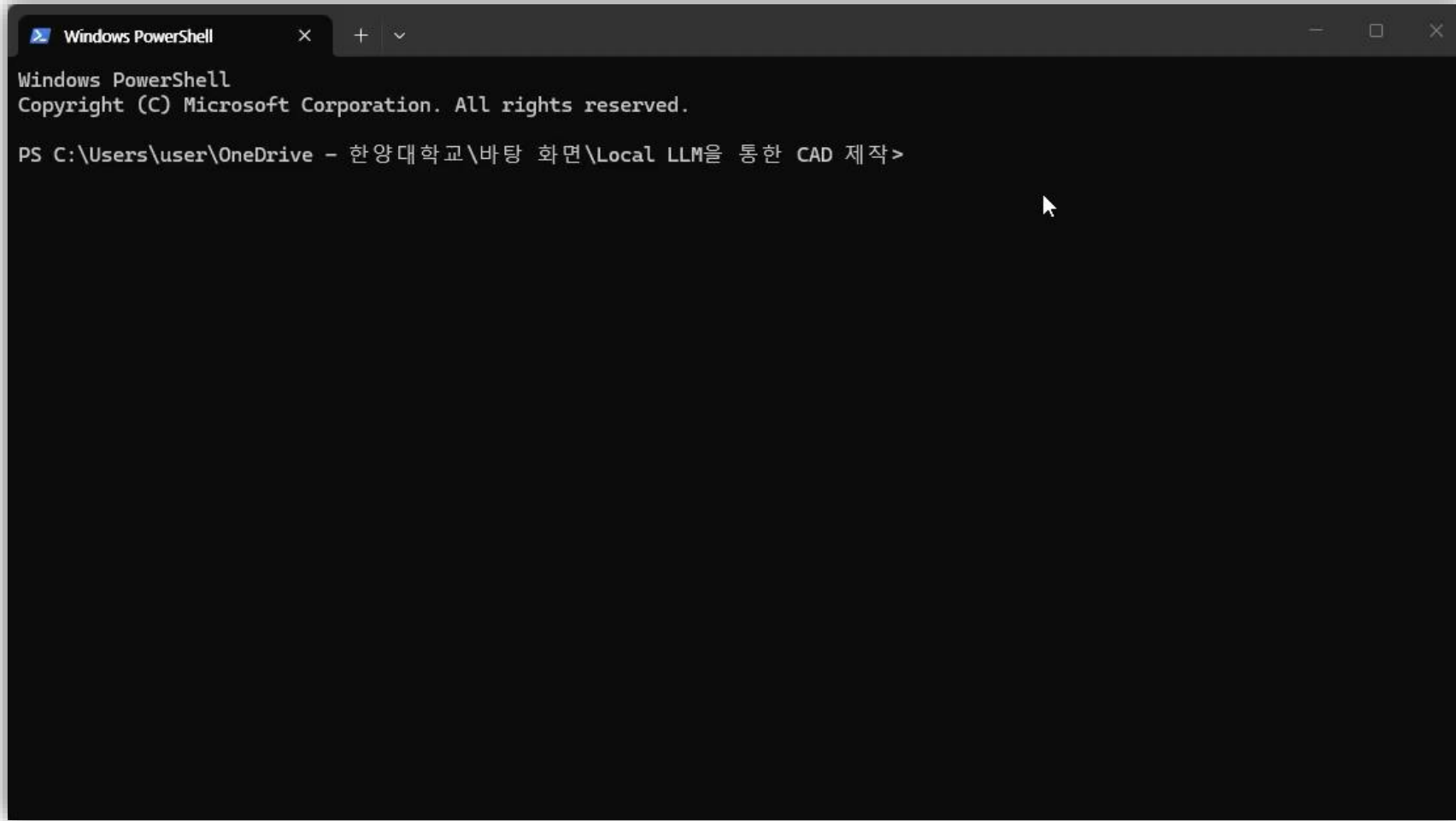
### 결론 6

에어포일 좌표는 LLM이 생성하는 것이 아닌, LLM은 NACA 번호와 설계 변수만 구조화하며 Python이 공식을 계산한다.

### 결론 7

최종 성공 판정은 “화면에 형상이 보임”이 아니라 아래의 기준을 따라야 한다.

```
계획 검증 통과
+ 원문 치수 일치
+ 컴파일 성공
+ Solid=1
+ isValid=True
+ 핵심 치수 일치
+ 파일 변환 후 형상 보존
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\user\OneDrive - 한양대학교\바탕 화면\Local LLM을 통한 CAD 제작>
```

# Any Questions?

이름 최준석  
소속 한양대학교 기계공학과  
이메일 chw10109@hanyang.ac.kr



*Intelligent Design  
for Engineering  
Applications Lab*

